UMRR96 T153 AUTOMOTIVE V1.2.2 USER INTERFACE

DATE:
February 16, 2026

USER INTERFACE NAME:
UMRR96 T153 AUTOMOTIVE

USER INTERFACE VERSION:
v1.2.2

s.m.s, smart microwave sensors GmbH
In den Waashainen 1
38108 Braunschweig
Germany

Phone: +49 531 39023-0
Fax: +49 531 39023-599
info@smartmicro.de
www.smartmicro.com

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

CONTENT

1 COMMUNICATION DATA STREAM SERVICE 2
1.1 COMTARGETBASELIST PORT 2
1.2 COMTARGETLIST PORT 4
2 LEGAL DISCLAIMER NOTICE 6
APPENDIX 7
A COMMUNICATION DATA SERVICE API 7
B USER INTERFACE INSTRUCTIONS UMRR96 T153 AUTOMOTIVE VERSION 1.2.2 15
B.1 Parameter Section auto_interface_0dim 15
B.2 Status Section auto_interface 20
B.3 Command Section auto_interface_command 22

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

1 COMMUNICATION DATA STREAM SERVICE

With the communication data stream service smartmicro ports can be received as C++ objects with simplified access functions, which are generated by the user interface. Smartmicro ports are data buffers which contains data recorded by the radar data: e.g objects, statistics, statuses of device etc. Each port contains a generic port header, with a port description : version, id, size etc. Sometime ports also contains dynamic list of objects. In order to receive a port, a callback needs to be registered with the service. The callback will be carried out periodically every sensor cycle time.

Please note:

- This callback will be called in the context of a receiver thread, so the data needs to be copied and the callback must be released. Otherwise, the reception will be blocked.
- It is possible to use one callback function for several clients with the same port and same user interface, but it is not allowed to use one callback function for different ports or different user interfaces.

For more details please see the examples below. The following ports are supported.

1.1 COMTARGETBASELIST PORT

Description:
To receive the port called "ComTargetBaseList" from a specific client, please use the following registration interface:

```
#include <umrr96_t153_automotive_v1_2_2/DataStreamServiceIface.h>

void ReceiveComTargetBaseListClbk(IN std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::
    comtargetbaselist::ComTargetBaseList> comTargetBaseList, com::types::ClientId clientId)
{
        // Getting members of  ComTargetBaseList
    std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::comtargetbaselist::PortHeader>
        portHeader =
                        comTargetBaseList->GetPortHeader();
    std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::comtargetbaselist::
        TargetListHeader> targetListHeader =
                        comTargetBaseList->GetTargetListHeader();
    auto targetList = comTargetBaseList->GetTargetList();
    // Getting members of  PortHeader
    std::cout << "Variable PortIdentifier:"
              << portHeader->GetPortIdentifier()
              << std::endl;
    std::cout << "Variable PortVersionMajor:"
              << portHeader->GetPortVersionMajor()
              << std::endl;
    std::cout << "Variable PortVersionMinor:"
              << portHeader->GetPortVersionMinor()
              << std::endl;
    std::cout << "Variable Timestamp:"
              << portHeader->GetTimestamp()
              << std::endl;
    std::cout << "Variable PortSize:"
              << portHeader->GetPortSize()
              << std::endl;
    std::cout << "Variable BodyEndianness:"
              << portHeader->GetBodyEndianness()
              << std::endl;
    std::cout << "Variable PortIndex:"
              << portHeader->GetPortIndex()
              << std::endl;
    std::cout << "Variable HeaderVersionMajor:"
              << portHeader->GetHeaderVersionMajor()
              << std::endl;
    std::cout << "Variable HeaderVersionMinor:"
              << portHeader->GetHeaderVersionMinor()
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
                << std::endl;
        // Getting members of  TargetListHeader
        std::cout << "Variable CycleDuration:"
                << targetListHeader->GetCycleDuration()
                << std::endl;
        std::cout << "Variable NumberOfTargets:"
                << targetListHeader->GetNumberOfTargets()
                << std::endl;
        std::cout << "Variable CycleCount:"
                << targetListHeader->GetCycleCount()
                << std::endl;
        std::cout << "Variable AcquisitionSetup:"
                << targetListHeader->GetAcquisitionSetup()
                << std::endl;
        std::cout << "Variable TimeStamp:"
                << targetListHeader->GetTimeStamp()
                << std::endl;
        std::cout << "Variable AcqTimeStampFraction:"
                << targetListHeader->GetAcqTimeStampFraction()
                << std::endl;
        // Getting members of  Target
        for(auto& target : targetList)
        {
            std::cout << "Variable Range:"
                      << target->GetRange()
                      << std::endl;
            std::cout << "Variable SpeedRadial:"
                      << target->GetSpeedRadial()
                      << std::endl;
            std::cout << "Variable AzimuthAngle:"
                      << target->GetAzimuthAngle()
                      << std::endl;
            std::cout << "Variable ElevationAngle:"
                      << target->GetElevationAngle()
                      << std::endl;
            std::cout << "Variable RCS:"
                      << target->GetRCS()
                      << std::endl;
            std::cout << "Variable SignalLevel:"
                      << target->GetSignalLevel()
                      << std::endl;
            std::cout << "Variable Noise:"
                      << target->GetNoise()
                      << std::endl;
        }

    }
        auto comDataStreamServ = com::master::umrr96_t153_automotive_v1_2_2::DataStreamServiceIface::
            Get();
        ClientId clientIdA = 1024; // client id from sensor a
        ClientId clientIdB = 1025; // client id from sensor b
        ReceiveComTargetBaseListCallback callback =
                                std::bind(&ReceiveComTargetBaseListClbk,
                                std::placeholders::_1,
                                std::placeholders::_2);
        if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetBaseListReceiveCallback(clientIdA,
            callback)
        {
            std::cout << "Failed to register ComTargetBaseList port callback" << std::endl;
        }
        if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetBaseListReceiveCallback(clientIdB,
            callback))
        {
            std::cout << "Failed to register ComTargetBaseList port callback" << std::endl;
        }
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

1.2 COMTARGETLIST PORT

Description:
To receive the port called "ComTargetList" from a specific client, please use the following registration interface:

```
#include <umrr96_t153_automotive_v1_2_2/DataStreamServiceIface.h>

void ReceiveComTargetListClbk(IN std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::
    comtargetlist::ComTargetList> comTargetList, com::types::ClientId clientId)
{
        // Getting members of  ComTargetList
    std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::comtargetlist::PortHeader>
        portHeader =
                        comTargetList->GetPortHeader();
    std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::comtargetlist::TargetListHeader>
        targetListHeader =
                        comTargetList->GetTargetListHeader();
    auto targetList = comTargetList->GetTargetList();
    // Getting members of  PortHeader
    std::cout << "Variable PortIdentifier:"
              << portHeader->GetPortIdentifier()
              << std::endl;
    std::cout << "Variable PortVersionMajor:"
              << portHeader->GetPortVersionMajor()
              << std::endl;
    std::cout << "Variable PortVersionMinor:"
              << portHeader->GetPortVersionMinor()
              << std::endl;
    std::cout << "Variable Timestamp:"
              << portHeader->GetTimestamp()
              << std::endl;
    std::cout << "Variable PortSize:"
              << portHeader->GetPortSize()
              << std::endl;
    std::cout << "Variable BodyEndianness:"
              << portHeader->GetBodyEndianness()
              << std::endl;
    std::cout << "Variable PortIndex:"
              << portHeader->GetPortIndex()
              << std::endl;
    std::cout << "Variable HeaderVersionMajor:"
              << portHeader->GetHeaderVersionMajor()
              << std::endl;
    std::cout << "Variable HeaderVersionMinor:"
              << portHeader->GetHeaderVersionMinor()
              << std::endl;
    // Getting members of  TargetListHeader
    std::cout << "Variable CycleTime:"
              << targetListHeader->GetCycleTime()
              << std::endl;
    std::cout << "Variable NumberOfTargets:"
              << targetListHeader->GetNumberOfTargets()
              << std::endl;
    std::cout << "Variable AcquisitionSetup:"
              << targetListHeader->GetAcquisitionSetup()
              << std::endl;
    // Getting members of  Target
    for(auto& target : targetList)
    {
        std::cout << "Variable Range:"
                  << target->GetRange()
                  << std::endl;
        std::cout << "Variable SpeedRadial:"
                  << target->GetSpeedRadial()
                  << std::endl;
        std::cout << "Variable AzimuthAngle:"
                  << target->GetAzimuthAngle()
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
                << std::endl;
        std::cout << "Variable ElevationAngle:"
                << target->GetElevationAngle()
                << std::endl;
        std::cout << "Variable VarianceRange:"
                << target->GetVarianceRange()
                << std::endl;
        std::cout << "Variable VarianceSpeed:"
                << target->GetVarianceSpeed()
                << std::endl;
        std::cout << "Variable VarianceAzimuthAngle:"
                << target->GetVarianceAzimuthAngle()
                << std::endl;
        std::cout << "Variable VarianceElevationAngle:"
                << target->GetVarianceElevationAngle()
                << std::endl;
        std::cout << "Variable RCS:"
                << target->GetRCS()
                << std::endl;
        std::cout << "Variable FalseAlarmProbability:"
                << target->GetFalseAlarmProbability()
                << std::endl;
        std::cout << "Variable Flags:"
                << target->GetFlags()
                << std::endl;
        std::cout << "Variable Power:"
                << target->GetPower()
                << std::endl;
        std::cout << "Variable Noise:"
                << target->GetNoise()
                << std::endl;
        std::cout << "Variable PeakIdx:"
                << target->GetPeakIdx()
                << std::endl;
    }

}
    auto comDataStreamServ = com::master::umrr96_t153_automotive_v1_2_2::DataStreamServiceIface::
        Get();
    ClientId clientIdA = 1024; // client id from sensor a
    ClientId clientIdB = 1025; // client id from sensor b
    ReceiveComTargetListCallback callback =
                        std::bind(&ReceiveComTargetListClbk,
                        std::placeholders::_1,
                        std::placeholders::_2);
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetListReceiveCallback(clientIdA,
        callback)
    {
        std::cout << "Failed to register ComTargetList port callback" << std::endl;
    }
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetListReceiveCallback(clientIdB,
        callback))
    {
        std::cout << "Failed to register ComTargetList port callback" << std::endl;
    }
```

For a more detailed API description, please see Appendix A.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

2 LEGAL DISCLAIMER NOTICE

All products, product specifications and data in this document may be subject to change without notice to improve reliability, function or otherwise. Not all products and/or product features may be available in all countries and regions. For legal reasons features may be deleted from products or smartmicro may refuse to offer products. Statements, technical information and recommendations contained herein are believed to be accurate as of the stated date. smartmicro disclaims any and all liability for any errors, inaccuracies or incompleteness contained in this document or in any other disclosure relating to the product.

To the extent permitted by applicable law, smartmicro disclaims (i) any and all liability arising out of the application or use of the product or the data contained herein, (ii) any and all liability of damages exceeding direct damages, including - without limitation - indirect, consequential or incidental damages, and (iii) any and all implied warranties, including warranties of the suitability of the product for particular purposes.

Statements regarding the suitability of products for certain types of applications are based on smartmicro's knowledge of typical requirements that are often placed on smartmicro products in generic/general applications. Statements about the suitability of products for a particular/specific application, however, are not binding. It is the customer's/user's responsibility to validate that the product with the specifications described is suitable for use in the particular/specific application. Parameters and the performance of products may deviate from statements made herein due to particular/specific applications and/or surroundings. Therefore, it is important that the customer/user has thoroughly tested the products and has understood the performance and limitations of the products before installing them for final applications or before their commercialization. Although products are well optimized to be used for the intended applications stated, it must also be understood by the customer/user that the detection probability may not be 100% and that the false alarm rate may not be zero.

The information provided, relates only to the specifically designated product and may not be applicable when the product is used in combination with other materials or in any process not defined herein. All operating parameters, including typical parameters, must be validated for each application by the customer's/user's technical experts. Customers using or selling smartmicro products for use in an application which is not expressly indicated do so at their own risk. This document does not expand or otherwise modify smartmicro's terms and conditions of purchase, including but not being limited to the warranty. Except as expressly indicated in writing by smartmicro, the products are not designed for use in medical, lifesaving or life-sustaining applications or for any other application in which the failure of the product could result in personal injury or death.

No license expressed or implied, by estoppel or otherwise, to any intellectual property rights is granted by this document or by any conduct of smartmicro. Product names and markings noted herein may be trademarks of their respective owners.

Please note that the application of the product may be subject to standards or other regulations that may vary from country to country. smartmicro does not guarantee that the use of products in the applications described herein will comply with such regulations in any country. It is the customer's/user's responsibility to ensure that the use and incorporation of products comply with regulatory requirements of their markets.

If any provision of this disclaimer is, or is found to be, void or unenforceable under applicable law, it will not affect the validity or enforceability of the other provisions of this disclaimer.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

A COMMUNICATION DATA SERVICE API

A **ComTargetBaseList**
Description: Base target list port for automotive
The object ComTargetBaseList provides the following APIs:

```
std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::PortHeader> GetPortHeader() const
    ;
```

Returns pointer to PortHeader object, whose access functions are described below:
A **PortHeader**
Description: smartmicro Generic Port Header
The object PortHeader provides the following APIs:

```
uint32_t GetPortIdentifier() const;
```

Returns value of PortIdentifier of uint32_t data type.
PortIdentifier - Unique port identifier

```
int16_t GetPortVersionMajor() const;
```

Returns value of PortVersionMajor of int16_t data type.
PortVersionMajor - Major port version

```
int16_t GetPortVersionMinor() const;
```

Returns value of PortVersionMinor of int16_t data type.
PortVersionMinor - Minor port version

```
uint64_t GetTimestamp() const;
```

Returns value of Timestamp of uint64_t data type.
Timestamp - A timestamp

```
uint32_t GetPortSize() const;
```

Returns value of PortSize of uint32_t data type.
PortSize - Port size (includes this header)

```
uint8_t GetBodyEndianness() const;
```

Returns value of BodyEndianness of uint8_t data type.
BodyEndianness - Port body endianness[1:big endian][2:little endian]

```
uint8_t GetPortIndex() const;
```

Returns value of PortIndex of uint8_t data type.
PortIndex - To identify multiple instances of the same port

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint8_t GetHeaderVersionMajor() const;
```

Returns value of HeaderVersionMajor of uint8_t data type.
HeaderVersionMajor - Generic port header major version

```
uint8_t GetHeaderVersionMinor() const;
```

Returns value of HeaderVersionMinor of uint8_t data type.
HeaderVersionMinor - Generic port header minor version

```
std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::TargetListHeader>
    GetTargetListHeader() const;
```

Returns pointer to TargetListHeader object, whose access functions are described below:
A **TargetListHeader**
Description: Static header of the target list
The object TargetListHeader provides the following APIs:

```
float32_t GetCycleTime() const;
```

Returns value of CycleTime of float32_t data type.
CycleTime - Cycle Time of the radar. Cycle time could vary depending on the mode of operation of the radar.

```
uint16_t GetNumberOfTargets() const;
```

Returns value of NumberOfTargets of uint16_t data type.
NumberOfTargets - Number of the detected targets.

```
uint16_t GetAcquisitionSetup() const;
```

Returns value of AcquisitionSetup of uint16_t data type.
AcquisitionSetup - Feedback channel of used Acquisition setup

```
const std::vector<std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::Target>>&
    GetTargetList() const;
```

Returns pointer to array of Target objects, whose access functions are described below:
A **Target**
Description: Represent a single target
The object Target provides the following APIs:

```
float32_t GetRange() const;
```

Returns value of Range of float32_t data type.
Range - Range of the detected target.

```
float32_t GetSpeedRadial() const;
```

Returns value of SpeedRadial of float32_t data type.
SpeedRadial - Radial speed of the detected target.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
float32_t GetAzimuthAngle() const;
```

Returns value of AzimuthAngle of float32_t data type.
AzimuthAngle - Azimuth angle of the detected target.

```
float32_t GetElevationAngle() const;
```

Returns value of ElevationAngle of float32_t data type.
ElevationAngle - Elevation angle of the detected target.

```
float32_t GetVarianceRange() const;
```

Returns value of VarianceRange of float32_t data type.
VarianceRange - Variance of the range.

```
float32_t GetVarianceSpeed() const;
```

Returns value of VarianceSpeed of float32_t data type.
VarianceSpeed - Variance of the speed.

```
float32_t GetVarianceAzimuthAngle() const;
```

Returns value of VarianceAzimuthAngle of float32_t data type.
VarianceAzimuthAngle - Variance of the azimuth angle.

```
float32_t GetVarianceElevationAngle() const;
```

Returns value of VarianceElevationAngle of float32_t data type.
VarianceElevationAngle - Variance of the elevation angle.

```
float32_t GetRCS() const;
```

Returns value of RCS of float32_t data type.
RCS - The radar cross section of the detected target.

```
float32_t GetFalseAlarmProbability() const;
```

Returns value of FalseAlarmProbability of float32_t data type.
FalseAlarmProbability - Probability of the false alarm.

```
uint32_t GetFlags() const;
```

Returns value of Flags of uint32_t data type.
Flags - Flags as bitwise OR operation of port_target_list_target_flags 'target flags'.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
float32_t GetPower() const;
```

Returns value of Power of float32_t data type.
Power - Power of the detected target in decibels.

```
float32_t GetNoise() const;
```

Returns value of Noise of float32_t data type.
Noise - Noise of the detected target in decibels.

```
uint16_t GetPeakIdx() const;
```

Returns value of PeakIdx of uint16_t data type.
PeakIdx - Index of the peak in the peak list port (PLP) port that this target has been generated from.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

A **ComTargetList**
Description: Target list port for automotive standard products.
The object ComTargetList provides the following APIs:

```
std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::PortHeader> GetPortHeader() const
;
```

Returns pointer to PortHeader object, whose access functions are described below:
A **PortHeader**
Description: smartmicro Generic Port Header
The object PortHeader provides the following APIs:

```
uint32_t GetPortIdentifier() const;
```

Returns value of PortIdentifier of uint32_t data type.
PortIdentifier - Unique port identifier

```
int16_t GetPortVersionMajor() const;
```

Returns value of PortVersionMajor of int16_t data type.
PortVersionMajor - Major port version

```
int16_t GetPortVersionMinor() const;
```

Returns value of PortVersionMinor of int16_t data type.
PortVersionMinor - Minor port version

```
uint64_t GetTimestamp() const;
```

Returns value of Timestamp of uint64_t data type.
Timestamp - A timestamp

```
uint32_t GetPortSize() const;
```

Returns value of PortSize of uint32_t data type.
PortSize - Port size (includes this header)

```
uint8_t GetBodyEndianness() const;
```

Returns value of BodyEndianness of uint8_t data type.
BodyEndianness - Port body endianness[1:big endian][2:little endian]

```
uint8_t GetPortIndex() const;
```

Returns value of PortIndex of uint8_t data type.
PortIndex - To identify multiple instances of the same port

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint8_t GetHeaderVersionMajor() const;
```

Returns value of HeaderVersionMajor of uint8_t data type.
HeaderVersionMajor - Generic port header major version

```
uint8_t GetHeaderVersionMinor() const;
```

Returns value of HeaderVersionMinor of uint8_t data type.
HeaderVersionMinor - Generic port header minor version

```
std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::TargetListHeader>
    GetTargetListHeader() const;
```

Returns pointer to TargetListHeader object, whose access functions are described below:
A **TargetListHeader**
Description: Static header of the target list
The object TargetListHeader provides the following APIs:

```
float32_t GetCycleTime() const;
```

Returns value of CycleTime of float32_t data type.
CycleTime - Cycle Time of the radar. Cycle time could vary depending on the mode of operation of the radar.

```
uint16_t GetNumberOfTargets() const;
```

Returns value of NumberOfTargets of uint16_t data type.
NumberOfTargets - Number of the detected targets.

```
uint16_t GetAcquisitionSetup() const;
```

Returns value of AcquisitionSetup of uint16_t data type.
AcquisitionSetup - Feedback channel of used Acquisition setup

```
const std::vector<std::shared_ptr<com::master::umrr96_t153_automotive_v1_2_2::Target>>&
    GetTargetList() const;
```

Returns pointer to array of Target objects, whose access functions are described below:
A **Target**
Description: Represent a single target
The object Target provides the following APIs:

```
float32_t GetRange() const;
```

Returns value of Range of float32_t data type.
Range - Range of the detected target.

```
float32_t GetSpeedRadial() const;
```

Returns value of SpeedRadial of float32_t data type.
SpeedRadial - Radial speed of the detected target.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
float32_t GetAzimuthAngle() const;
```

Returns value of AzimuthAngle of float32_t data type.
AzimuthAngle - Azimuth angle of the detected target.

```
float32_t GetElevationAngle() const;
```

Returns value of ElevationAngle of float32_t data type.
ElevationAngle - Elevation angle of the detected target.

```
float32_t GetVarianceRange() const;
```

Returns value of VarianceRange of float32_t data type.
VarianceRange - Variance of the range.

```
float32_t GetVarianceSpeed() const;
```

Returns value of VarianceSpeed of float32_t data type.
VarianceSpeed - Variance of the speed.

```
float32_t GetVarianceAzimuthAngle() const;
```

Returns value of VarianceAzimuthAngle of float32_t data type.
VarianceAzimuthAngle - Variance of the azimuth angle.

```
float32_t GetVarianceElevationAngle() const;
```

Returns value of VarianceElevationAngle of float32_t data type.
VarianceElevationAngle - Variance of the elevation angle.

```
float32_t GetRCS() const;
```

Returns value of RCS of float32_t data type.
RCS - The radar cross section of the detected target.

```
float32_t GetFalseAlarmProbability() const;
```

Returns value of FalseAlarmProbability of float32_t data type.
FalseAlarmProbability - Probability of the false alarm.

```
uint32_t GetFlags() const;
```

Returns value of Flags of uint32_t data type.
Flags - Flags as bitwise OR operation of port_target_list_target_flags 'target flags'.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
float32_t GetPower() const;
```

Returns value of Power of float32_t data type.
Power - Power of the detected target in decibels.

```
float32_t GetNoise() const;
```

Returns value of Noise of float32_t data type.
Noise - Noise of the detected target in decibels.

```
uint16_t GetPeakIdx() const;
```

Returns value of PeakIdx of uint16_t data type.
PeakIdx - Index of the peak in the peak list port (PLP) port that this target has been generated from.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

B USER INTERFACE INSTRUCTIONS UMRR96 T153 AUTOMOTIVE VERSION 1.2.2

B.1 Parameter Section auto_interface_0dim

Automotive user interface 0dimensional parameters

Parameter Name	tx_antenna_idx
Description	Index of Transmit Antenna
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	center_frequency_idx
Description	Index of center frequency
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	2

Parameter Name	frequency_sweep_idx
Description	Index of sweep (0=226MHz, 1=512MHz, 2=1536MHz)
Data Type	u8
Dimensions	None
Access	RW
Default	2
Min	0
Max	2

Parameter Name	range_toggle_mode
Description	Automatic toggle of range:0=off, 1=Short-Med, 2=Short-Long, 3=Med-Long
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	3

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	prf_selector_manual
Description	0 = PRF switching active, 1 = PRF index given in prf_selector_index is used
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	1

Parameter Name	prf_set_selector
Description	Select PRF set index (only one set supported)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	0

Parameter Name	prf_manual_value_idx
Description	In manual PRF mode only: use nth element of selected set
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	tv_min_speed_sweep_idx_0
Description	Target Validation: minimum speed of target at first sweep
Data Type	f32
Dimensions	None
Access	RW
Default	-20.0
Min	-150.0
Max	150.0

Parameter Name	tv_min_speed_sweep_idx_1
Description	Target Validation: minimum speed of target at second sweep
Data Type	f32
Dimensions	None
Access	RW
Default	-20.0
Min	-150.0
Max	150.0

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	tv_min_speed_sweep_idx_2
Description	Target Validation: minimum speed of target at third sweep
Data Type	f32
Dimensions	None
Access	RW
Default	-20.0
Min	-150.0
Max	150.0

Parameter Name	tv_max_speed_sweep_idx_0
Description	Target Validation: maximum speed of target at first sweep
Data Type	f32
Dimensions	None
Access	RW
Default	20.0
Min	-150.0
Max	150.0

Parameter Name	tv_max_speed_sweep_idx_1
Description	Target Validation: maximum speed of target at second sweep
Data Type	f32
Dimensions	None
Access	RW
Default	20.0
Min	-150.0
Max	150.0

Parameter Name	tv_max_speed_sweep_idx_2
Description	Target Validation: maximum speed of target at third sweep
Data Type	f32
Dimensions	None
Access	RW
Default	20.0
Min	-150.0
Max	150.0

Parameter Name	output_control_target_list_can
Description	send raw targets via CAN, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	output_control_object_list_can
Description	send objects via CAN, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	0

Parameter Name	output_control_target_list_eth
Description	send raw targets via Ethernet, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_object_list_eth
Description	send objects via Ethernet, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	0

Parameter Name	ip_source_address
Description	IP source address (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	3232238347

Parameter Name	subnet_mask
Description	Subnet mask (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	4294967040

Parameter Name	ip_dest_address
Description	IP destination address (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	3232238353

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	ip_dest_port
Description	IP destination port
Data Type	u16
Dimensions	None
Access	RW
Default	55555

Parameter Name	sync_mode
Description	(Master+Slave config) 0=off, 1=master, 2=slave
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	sync_slave_identifier
Description	(Slave config) Unique Sync Slave Identifier, ignored on master (always 0)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	7

Parameter Name	sync_group_identifier
Description	(Slave config) Sync Group Identifier, ignored on master (always 0)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	1

Parameter Name	sync_nof_devices_1st_group
Description	(Master config) Number of synced devices (incl. master) in first group, ignored on slave
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	1
Max	8

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	sync_nof_devices_2nd_group
Description	(Master config) Number of synced devices in second group, ignored on slave
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	7

Parameter Name	sync_interface
Description	(Master+Slave config) interface for sensor sync, 1=can,2=ethernet
Data Type	u8
Dimensions	None
Access	RW
Default	2
Min	1
Max	2

Parameter Name	time_sync_mode
Description	(Time Sync: Master+Slave config) 0=off, 1=master, 2=slave
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	time_sync_nof_devices
Description	(Time Sync: Master config) Number of time synced devices (incl. master), ignored on slave
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	1
Max	8

B.2 Status Section auto_interface

customer status section

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Status Name	auto_interface_version_major
Description	Automotive interface version major. Increased, if new version is not totally backward compatible.
Data Type	u32
Dimensions	None
Access	R

Status Name	auto_interface_version_minor
Description	Automotive interface version minor. Increased, if parameters or statuses are changed or added. The new version is still backward compatible.
Data Type	u32
Dimensions	None
Access	R

Status Name	sw_generation
Description	Software Version generation
Data Type	u16
Dimensions	None
Access	R

Status Name	sw_version_major
Description	Software Version major
Data Type	u16
Dimensions	None
Access	R

Status Name	sw_version_minor
Description	Software Version minor
Data Type	u16
Dimensions	None
Access	R

Status Name	sw_version_patch
Description	Software Version patch
Data Type	u16
Dimensions	None
Access	R

Status Name	customer_id
Description	Customer Identifier
Data Type	u16
Dimensions	None
Access	R

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Status Name	product_serial
Description	32Bit product id serial
Data Type	u32
Dimensions	None
Access	R

Status Name	product_gen
Description	product generation
Data Type	u32
Dimensions	None
Access	R

Status Name	product_mod_high
Description	product modification high
Data Type	u32
Dimensions	None
Access	R

Status Name	product_mod_low
Description	product modification low
Data Type	u32
Dimensions	None
Access	R

Status Name	product_rev
Description	product revision
Data Type	u32
Dimensions	None
Access	R

Status Name	systime_low_dword
Description	time since the system timer was started in micro seconds (low dword)
Data Type	u32
Dimensions	None
Access	R

Status Name	systime_high_dword
Description	time since the system timer was started in micro seconds (high dword)
Data Type	u32
Dimensions	None
Access	R

B.3 Command Section auto_interface_command

Maintain compatible section 1000 commands

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Command Name	comp_sensor_reset_delayed_app_start
Description	Perform a device reset and stay the given value [seconds] in the bootloader at next startup (3074.2)

Command Name	comp_fsm_core0_opmode
Description	Select top level FSM operation mode (3078.1)

Command Name	comp_eeprom_ctrl_factory_reset
Description	Performs factory reset (3075.4)

Command Name	comp_sensor_reset
Description	Reset command which starts from BIOS (if available) or bootloader (3074.1)

Command Name	comp_pdi_requestor_can
Description	Send PDI data to client (3076.1)

Command Name	comp_eeprom_ctrl_save_param_sec
Description	Save the parameter inside the EEPROM. (3075.3)

Command Name	comp_eeprom_ctrl_reset_param_sec
Description	Restore default values in RAM. EEPROM content is not changed. (3075.2)

Command Name	comp_eeprom_ctrl_default_param_sec
Description	Restore default values in RAM and EEPROM. (3075.1)

Command Name	comp_timebase_set_seconds_val
Description	Set SECONDS value of NTP UTC timestamp

Command Name	comp_timebase_set_frac_seconds_val
Description	Set FRACTION_SECONDS value of NTP UTC timestamp

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.